

实时金融交易监控系统部署与开发指南

技术栈：Docker / Apache Kafka (KRaft) / Apache Spark (Structured Streaming) / MySQL / Grafana

本指南整合了构建实时端到端金融交易欺诈/可疑行为监控系统的完整基础设施脚本与核心处理代码。整体架构采用高效的流处理管道，能够实时捕捉高风险交易并将审计结果持久化存储。

第一阶段：基础设施搭建（Docker 环境）

请在终端中依次执行以下命令，创建自定义虚拟网络，并完成数据库、消息队列以及可视化大屏的容器化部署：

```
# 1. 创建统一的 Docker 自定义网络
docker network create bank-net

# 2. 启动 MySQL 数据库容器（映射到宿主机 3307 端口）
docker run --name BankingSQL -e MYSQL_ROOT_PASSWORD=private --network bank-net -p 3307:3306 -d mysql:8.0

# 3. 启动 Apache Kafka 容器（KRaft 模式，支持内外双通道访问）
docker run -d --name mykafka --network bank-net -p 9094:9094 -e KAFKA_NODE_ID=1 -e KAFKA_PROCESS_ROLES=broker,controller -e KAFKA_CONTROLLER_QUORUM_VOTERS=1@mykafka:9093 -e KAFKA_LISTENERS=INTERNAL://:9092,EXTERNAL://:9094,CONTROLLER://:9093 -e KAFKA_ADVERTISED_LISTENERS=INTERNAL://mykafka:9092,EXTERNAL://localhost:9094 -e KAFKA_LISTENER_SECURITY_PROTOCOL_MAP=INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT,CONTROLLER:PLAINTEXT -e KAFKA_INTER_BROKER_LISTENER_NAME=INTERNAL -e KAFKA_CONTROLLER_LISTENER_NAMES=CONTROLLER -e KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1 -e KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR=1 -e KAFKA_TRANSACTION_STATE_LOG_MIN_ISR=1 -e KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS=0 -e KAFKA_AUTO_CREATE_TOPICS_ENABLE=true apache/kafka:3.7.0

# 4. 在 Kafka 中手动创建交易主题管道（Topic）
docker exec -it mykafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server mykafka:9092 --create --topic bank-transactions --partitions 1 --replication-factor 1

# 5. 启动 Grafana 可视化监控大屏（本地访问 http://localhost:3000）
docker run -d --name grafana-bank --network bank-net -p 3000:3000 grafana/
```

第二阶段：MySQL 初始化建表

连接到已启动的 MySQL 数据库（宿主机访问地址 **127.0.0.1:3307**，用户名为 **root**，密码为 **private**），并在您的目标数据库（Schema）下执行以下 SQL 语句以完成审计表结构初始化：

```
CREATE TABLE suspicious_transactions
(
    id                INT AUTO_INCREMENT PRIMARY KEY,
    transaction_id    VARCHAR(100)          NULL,
    account_id        VARCHAR(100)          NULL,
    amount            DOUBLE                 NULL,
    transaction_type   VARCHAR(50)           NULL,
    country           VARCHAR(100)           NULL,
    status            VARCHAR(50)           NULL,
    created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP NULL
);
```

第三阶段：编写 Spark 实时流处理脚本 (span.py)

以下是使用 PySpark Structured Streaming 编写的流处理核心业务脚本。该脚本实现了从 Kafka 实时订阅交易数据流、动态解析 JSON 载荷、应用反欺诈规则过滤，并通过 JDBC 驱动以微批次（Micro-Batch）形式异步写入 MySQL：

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json
from pyspark.sql.types import StructType, StructField, StringType, DoubleType

# 1. 初始化 Spark 会话
spark =
SparkSession.builder      .appName("BankSuspiciousTransactionDetector")      .getOrCreate()

# 2. 定义从 Kafka 传输过来的交易数据的 JSON 拓扑结构
schema = StructType([
    StructField("transaction_id", StringType(), True),
    StructField("account_id", StringType(), True),
    StructField("amount", DoubleType(), True),
    StructField("transaction_type", StringType(), True),
    StructField("country", StringType(), True),
    StructField("status", StringType(), True)
])

# 3. 从 Kafka 实时流式读取数据 (外置 Spark 驱动连接宿主映射端口 9094)
kafka_stream_df =
spark.readStream      .format("kafka")      .option("kafka.bootstrap.servers",
"localhost:9094")      .option("subscribe", "bank-
transactions")      .option("startingOffsets", "latest")      .load()

# 4. 解析数据载荷: 将二进制 value 转换为字符串并投射到目标 Schema
parsed_df = kafka_stream_df      .selectExpr("CAST(value AS STRING) as
json_str")      .select(from_json(col("json_str"),
schema).alias("data"))      .select("data.*")

# 5. 【核心业务逻辑规则】实时过滤出触发风控标签的可疑交易
# 规则示例: 金额大于 10,000 或已经被上游系统标记为 FLAGGED
suspicious_df = parsed_df.filter(
    (col("amount") > 10000.0) | (col("status") == "FLAGGED")
)

# 6. 定义内部微批次沉淀函数, 将流数据写入关系型数据库
def write_to_mysql(df, epoch_id):
    # 注意: 请将 YOUR_DB_NAME 替换为您在 MySQL 中实际创建的数据库名称
    db_url = "jdbc:mysql://localhost:3307/YOUR_DB_NAME?
useSSL=false&allowPublicKeyRetrieval=true"
    df.write      .format("jdbc")      .option("url",
db_url)      .option("dbtable", "suspicious_transactions")      .option("user",
"root")      .option("password", "private")      .option("driver",
"com.mysql.cj.jdbc.Driver")      .mode("append")      .save()

```

```
# 7. 启动流式查询处理器
query =
suspicious_df.writeStream      .foreachBatch(write_to_mysql)      .outputMode("update")
    .start()

query.awaitTermination()
```

第四阶段：依赖安装与作业提交

在宿主机开发环境中，请先通过 pip 包管理器安装运行所需的客户端驱动，随后利用 **spark-submit** 命令将流处理作业分发至 Spark 计算引擎：

```
# 1. 安装本地 Python 依赖库
python3 -m pip install kafka-python pyspark

# 2. 提交并运行作业（执行时将自动在线检索并动态注入 Kafka 和 MySQL 的连接插件）
# 注意：请根据您本地实际安装的 Spark 环境版本，选择适配的插件连接版本（此处以 Spark 3.5.0 为例）
spark-submit --packages org.apache.spark:spark-sql-
kafka-0-10_2.12:3.5.0,mysql:mysql-connector-java:8.0.33 span.py
```